

Mouse Biometric Authentication System

A continuous behavioral authentication system using mouse movement dynamics for secure access control.

Table of Contents

- [Overview](#)
- [System Architecture](#)
- [Feature Taxonomy](#)
- [System Flowchart](#)
- [Installation](#)
- [Usage](#)
- [Technical Specifications](#)
- [Performance Metrics](#)

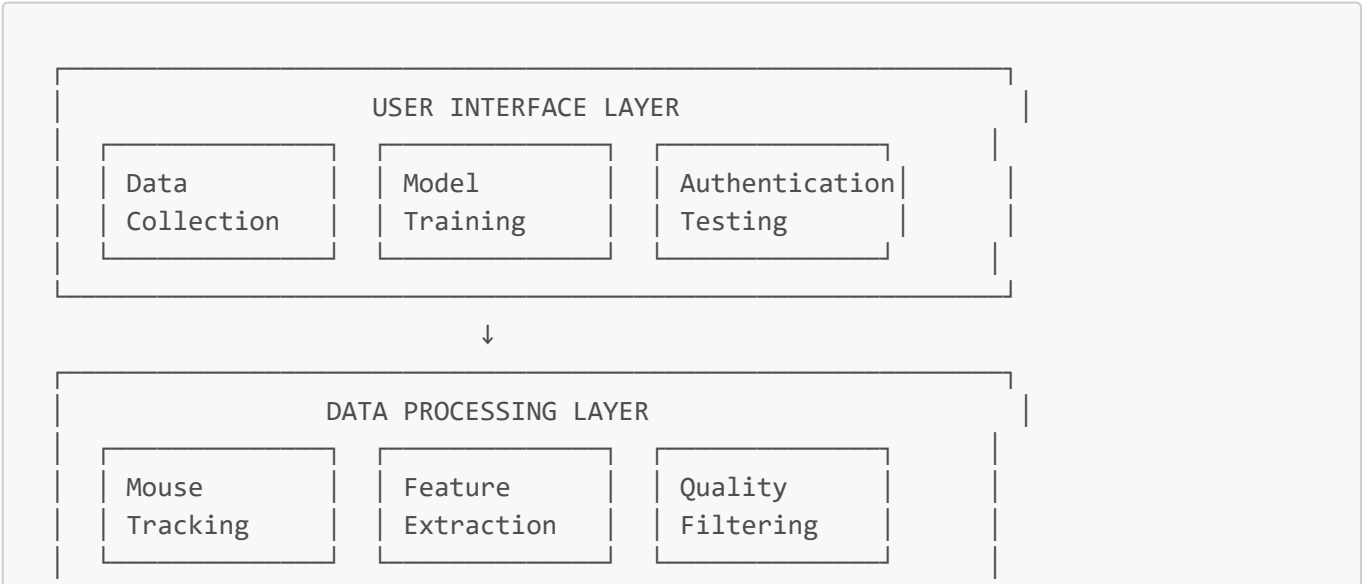
Overview

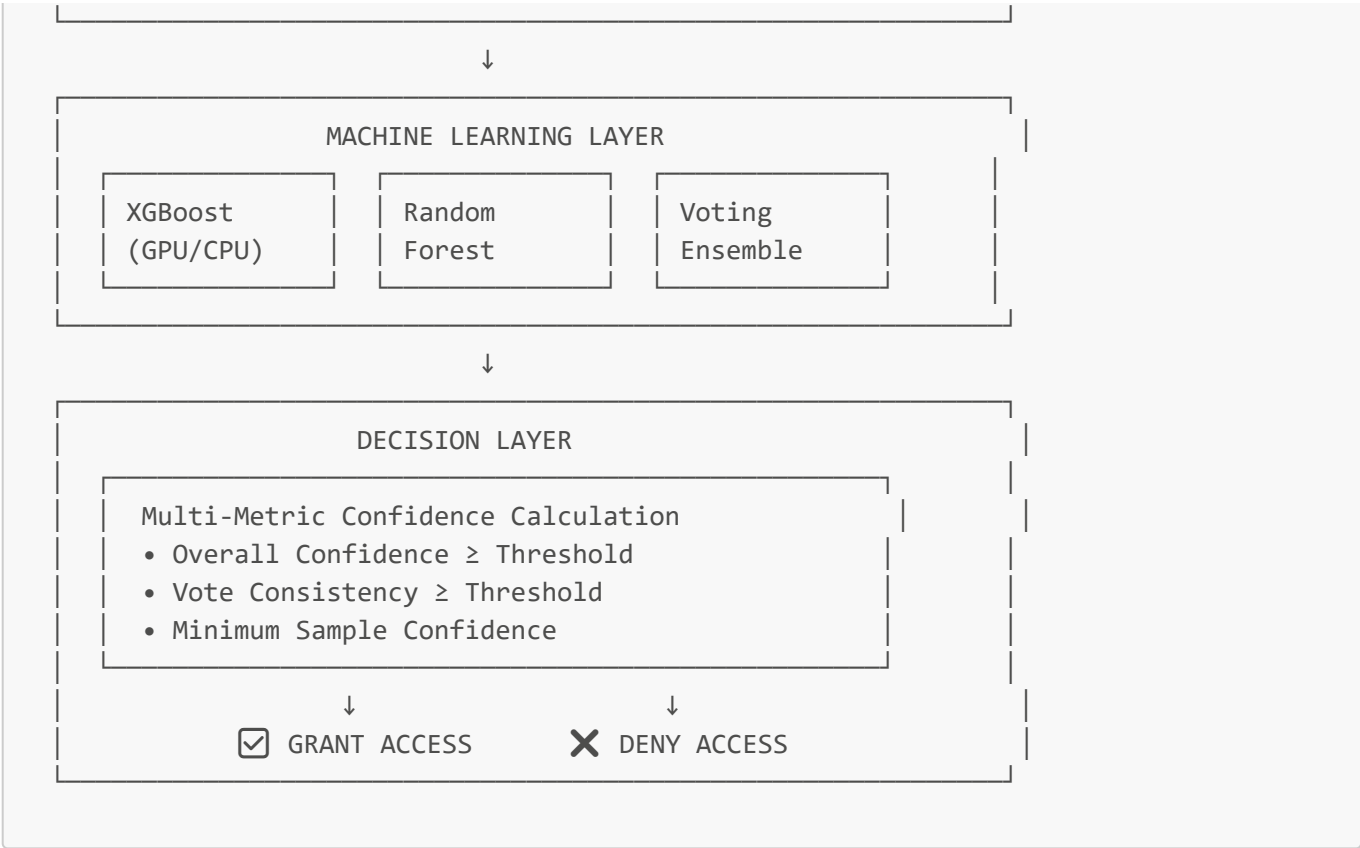
This project implements a **behavioral biometric authentication system** that identifies users based on their unique mouse movement patterns. Unlike traditional password-based systems that authenticate only at login, this system provides **continuous authentication** throughout a user session.

Key Features

- ☒ Non-intrusive behavioral biometrics
- ☒ Continuous authentication capability
- ☒ Multi-level security (Low/Medium/High)
- ☒ GPU-accelerated machine learning
- ☒ Professional dark-mode UI
- ☒ Real-time confidence scoring

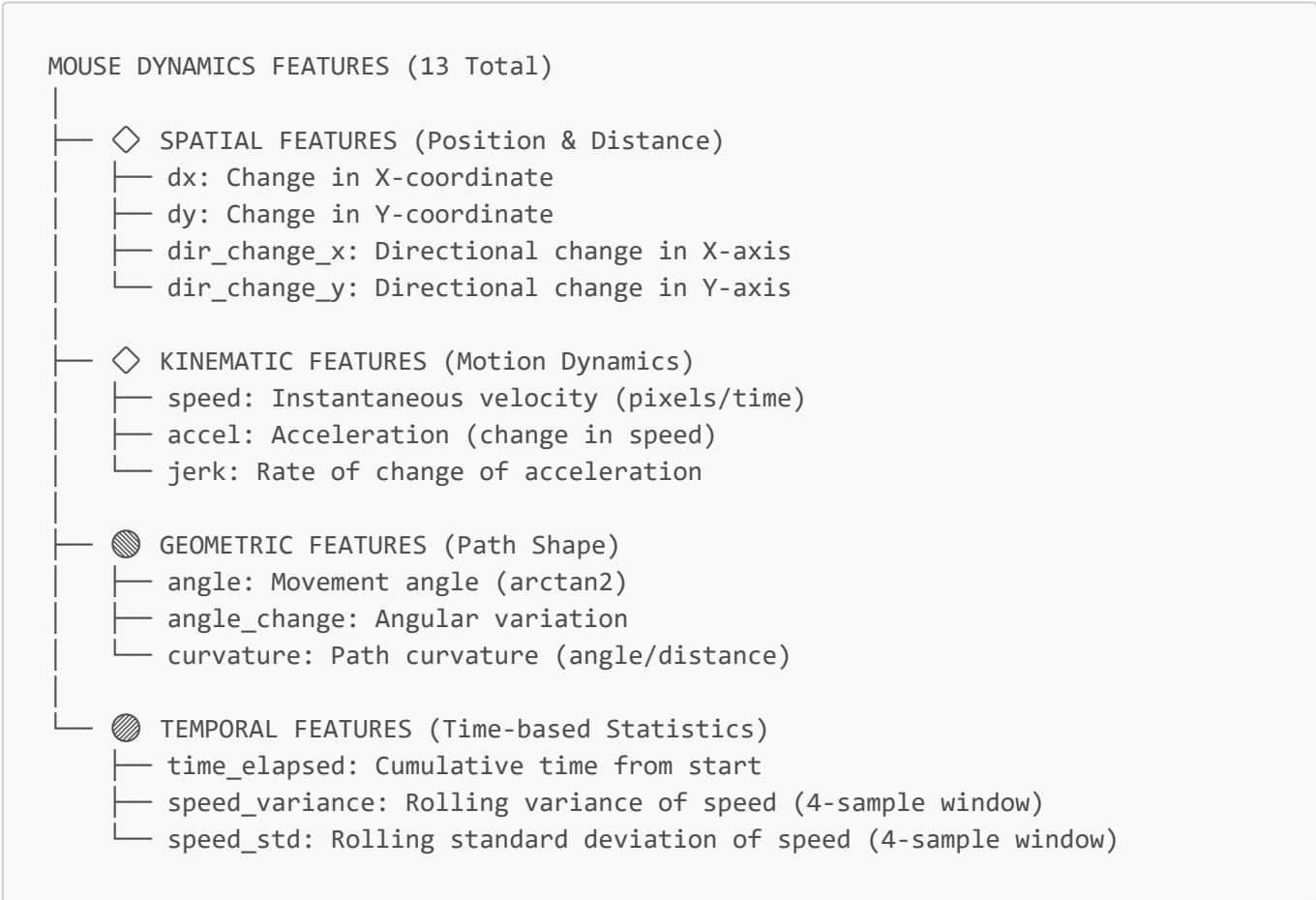
System Architecture



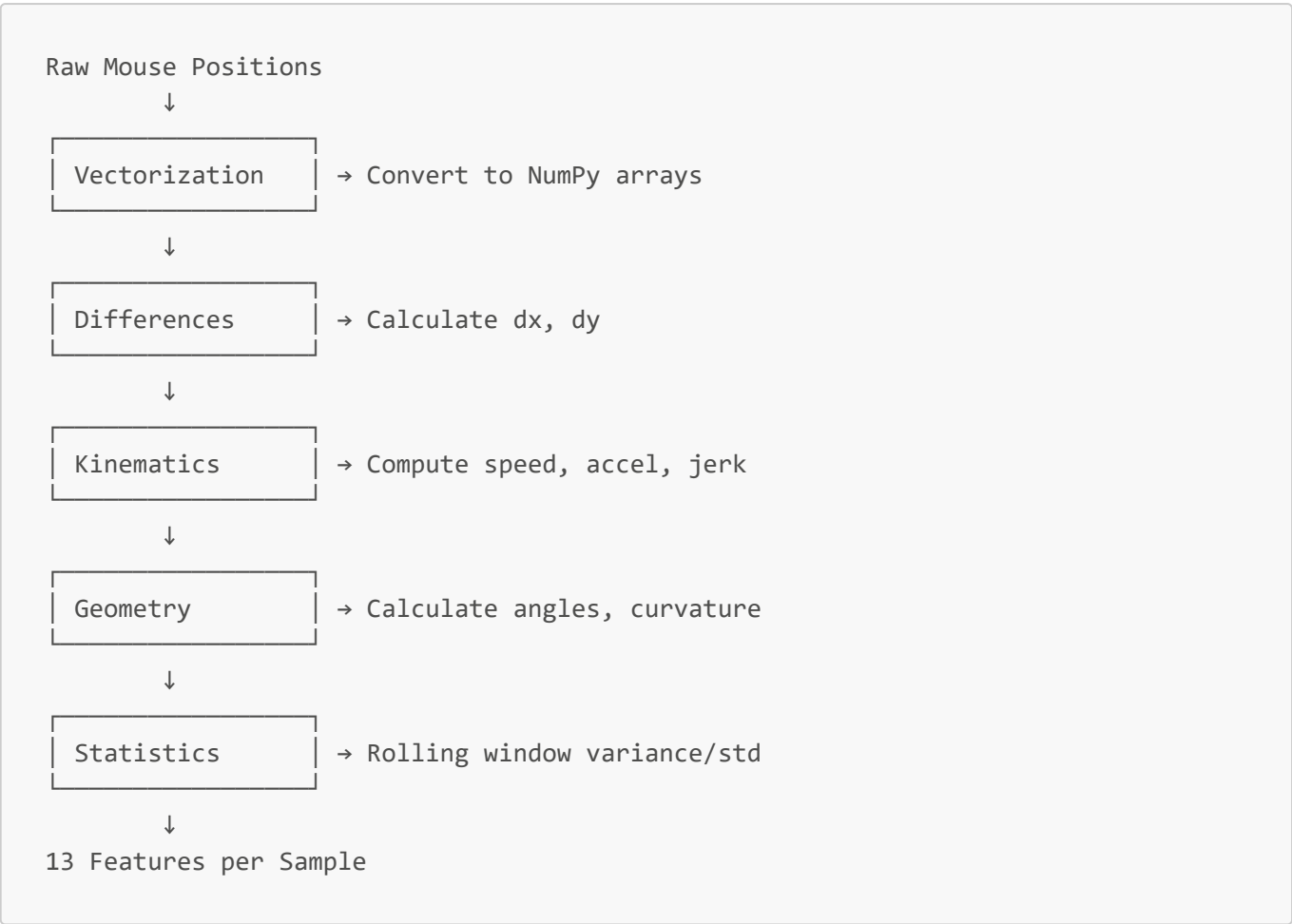


Feature Taxonomy

Feature Categories

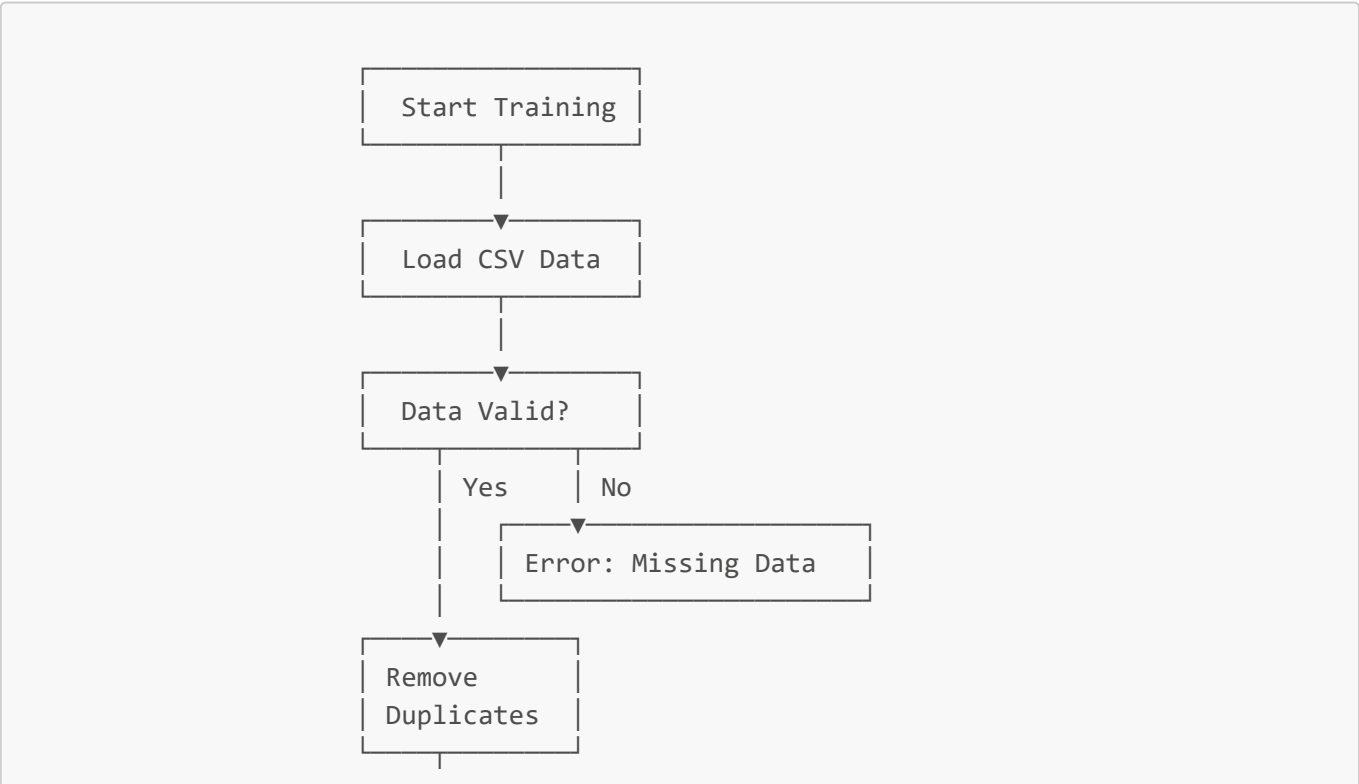


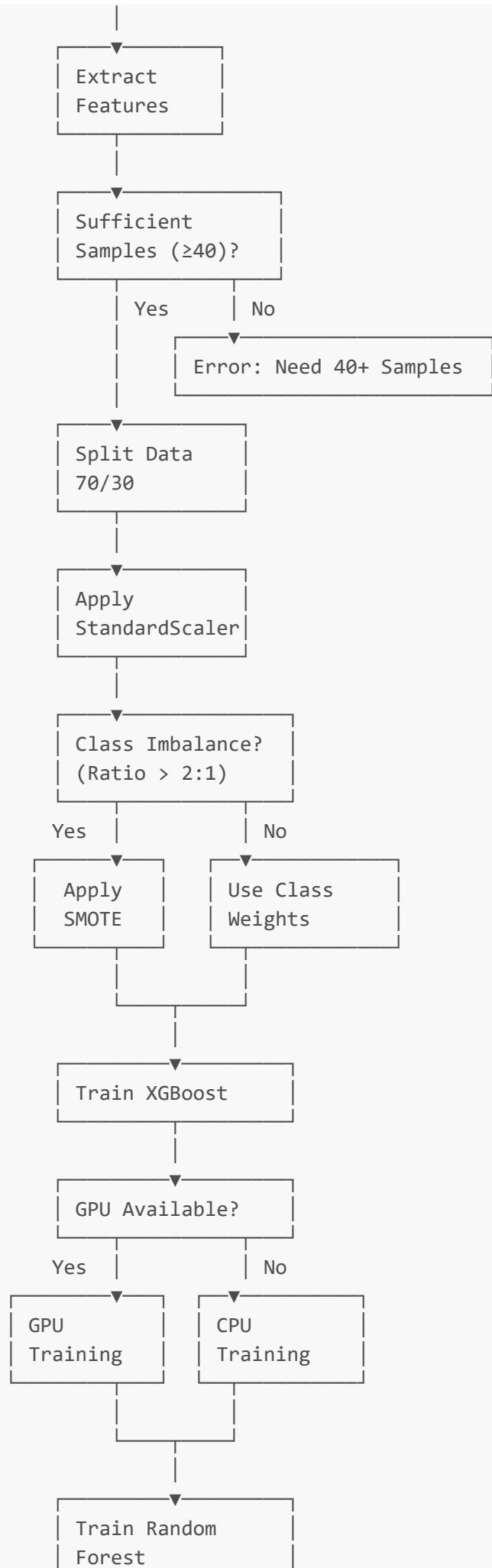
Feature Extraction Pipeline

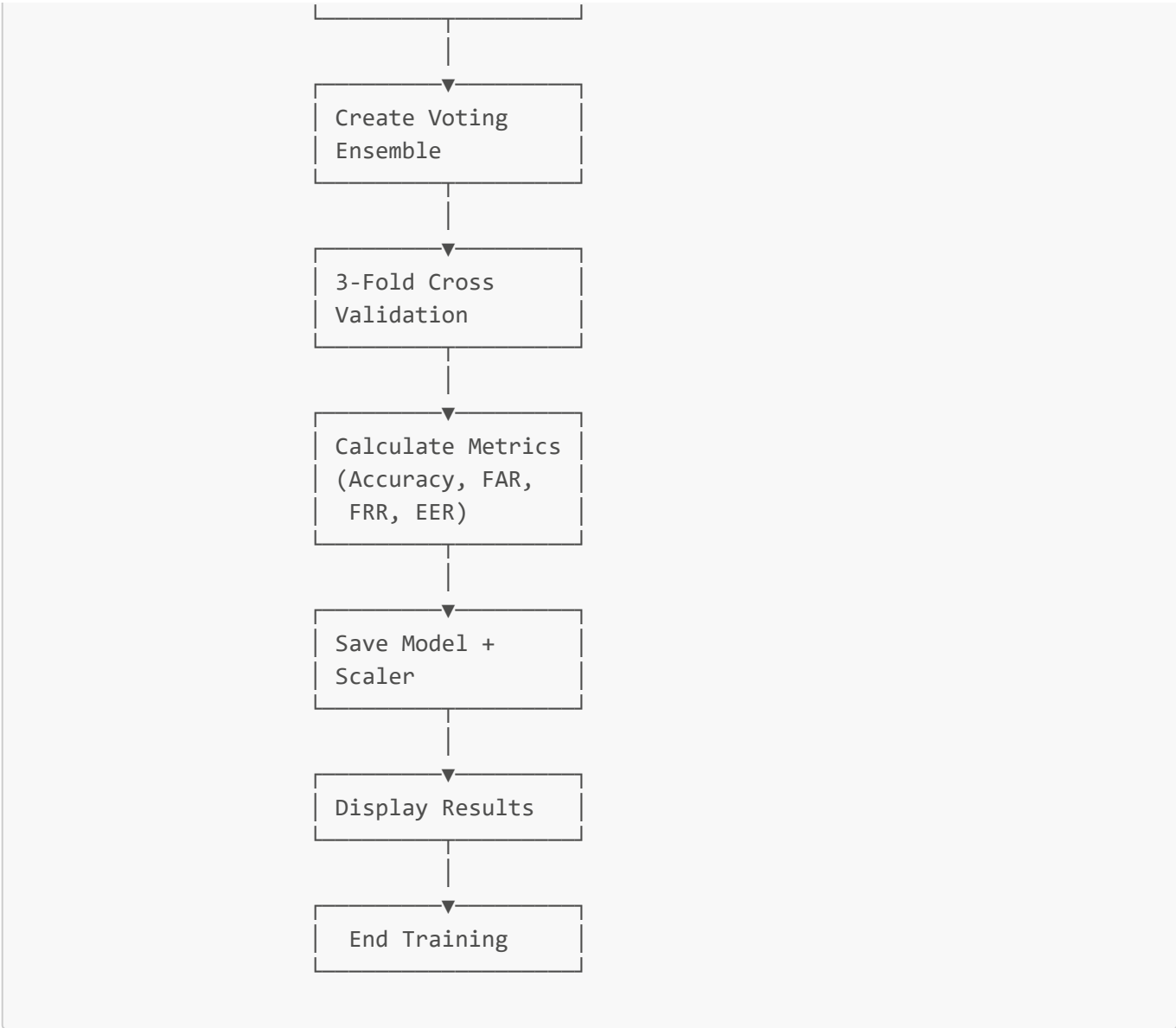


🔄 System Flowchart

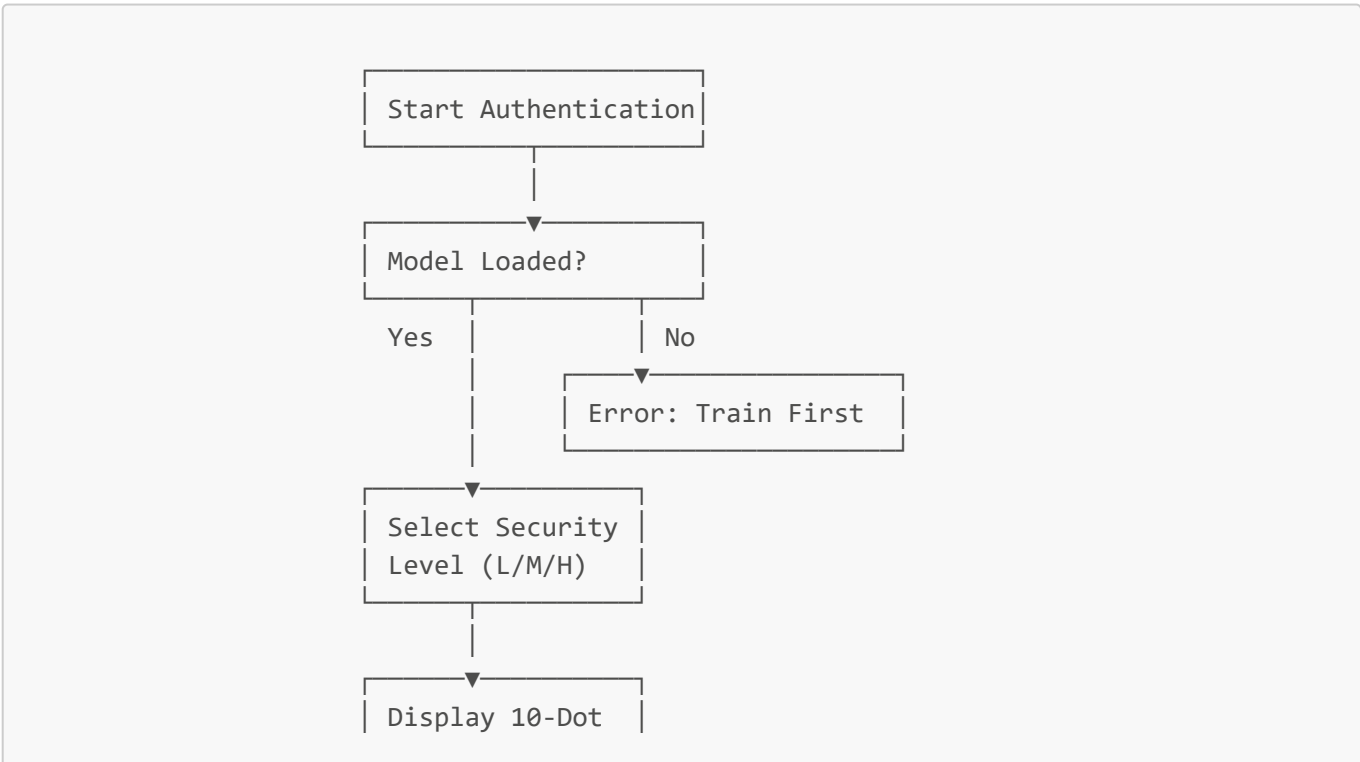
Training Phase

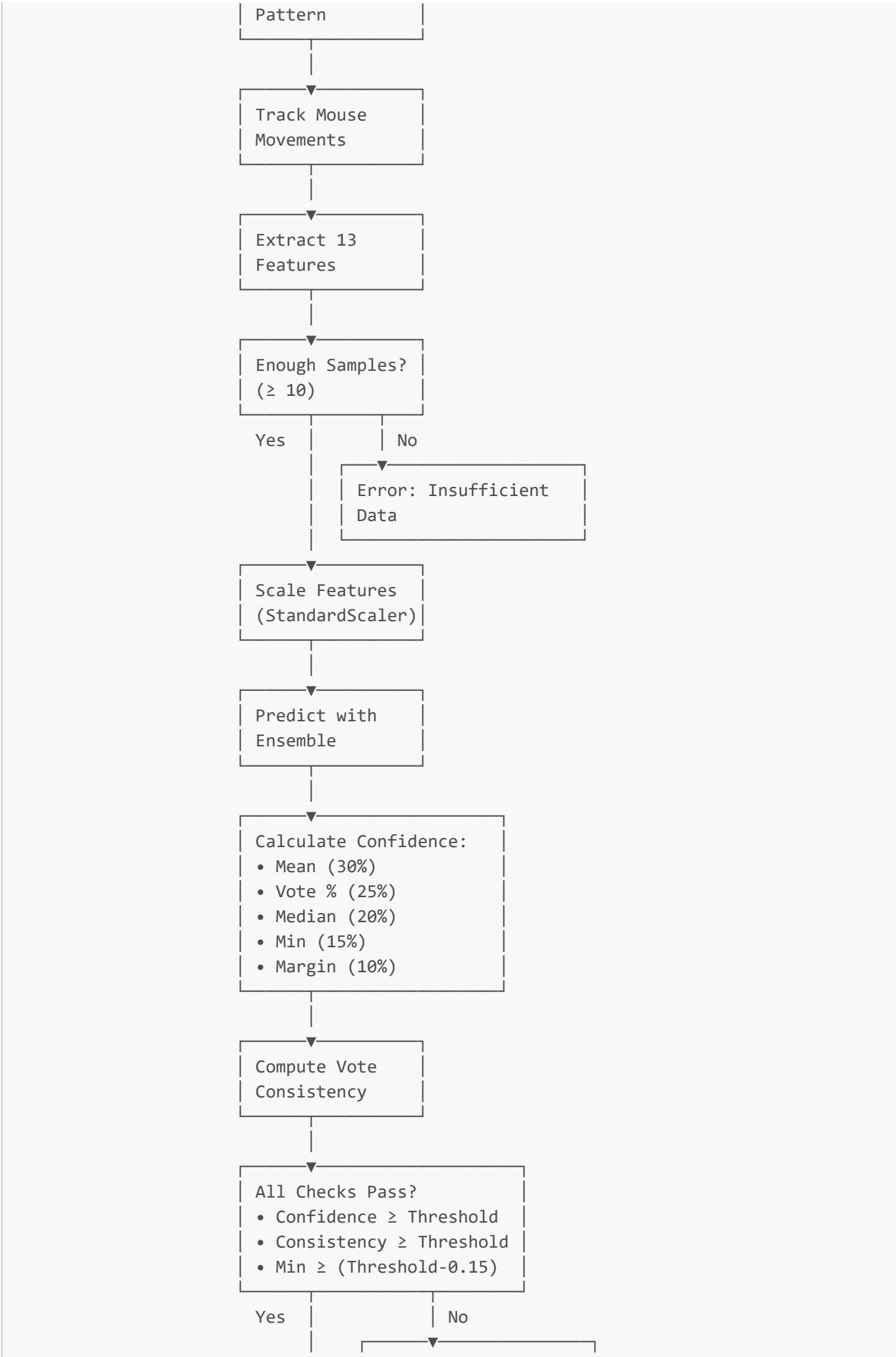


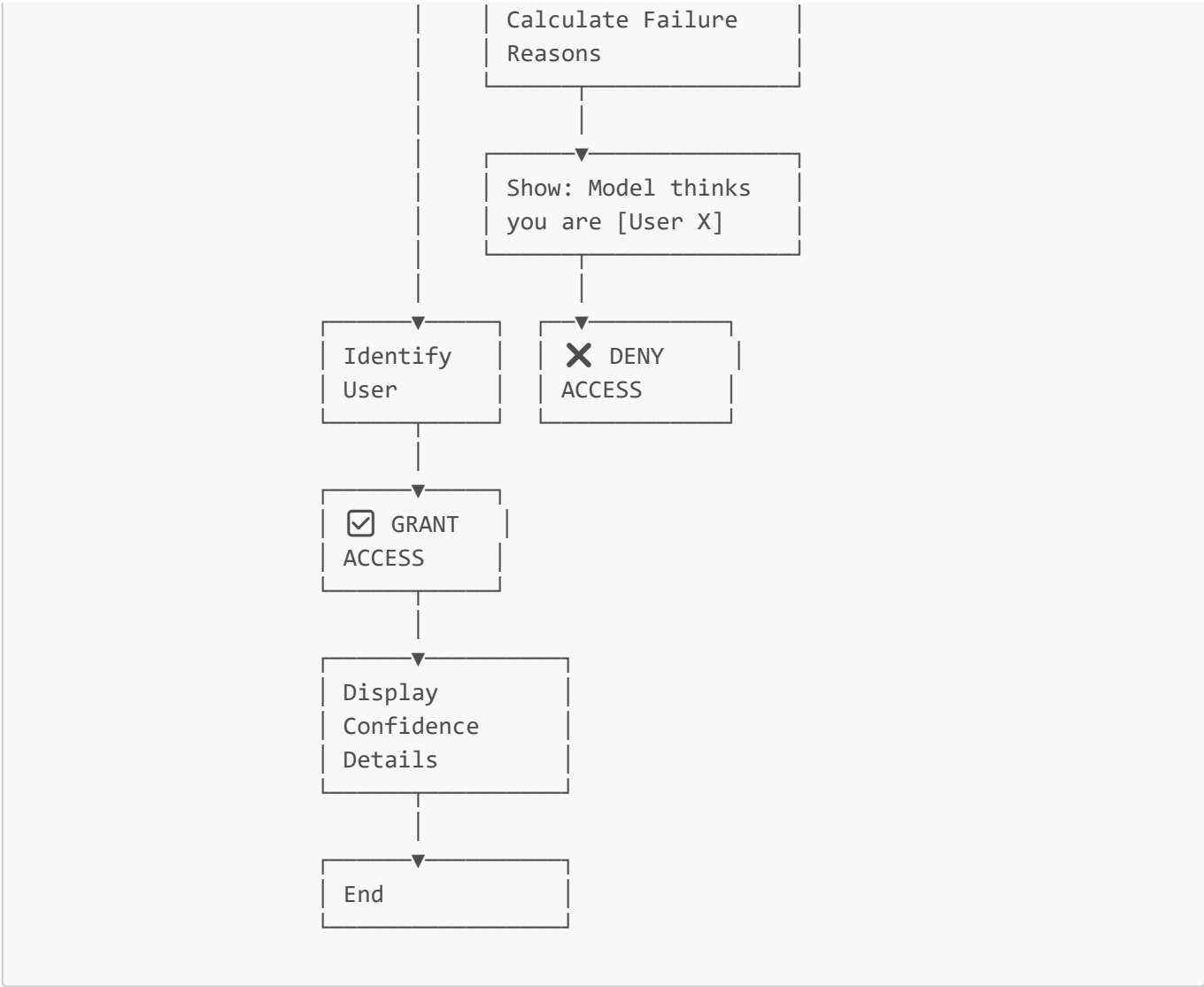




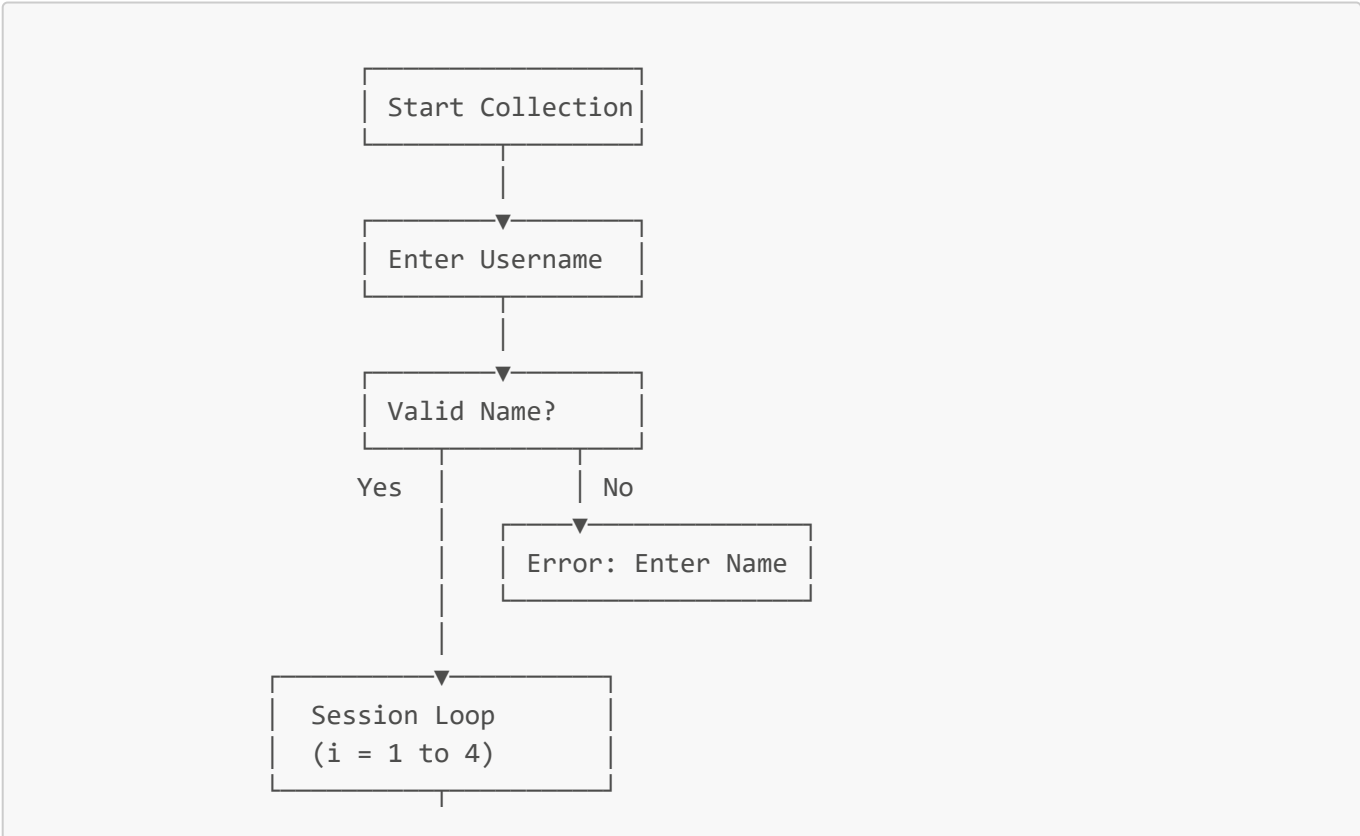
Authentication Phase

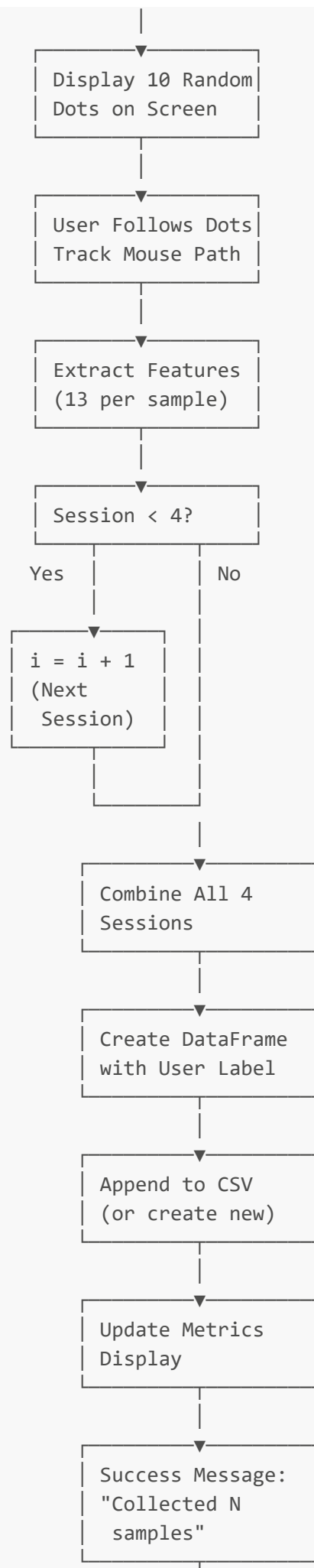






Data Collection Phase





End Collection

Installation

Prerequisites

- Python 3.10+
- Conda (Miniconda or Anaconda)
- Windows OS (current implementation)

Setup

1. Clone or download the project:

```
cd "C:\Users\Jonathan\Desktop\AAST\Cyper Physical System\Project"
```

2. The conda environment is already set up at:

```
C:\Users\Jonathan\Desktop\AAST\Cyper Physical System\Project\mouse
```

3. Run the application:

```
run.bat
```

Dependencies

The environment includes:

- **pandas** - Data manipulation
- **numpy** - Numerical computing
- **scikit-learn 1.7.2** - ML algorithms
- **xgboost** - Gradient boosting (GPU/CPU)
- **imbalanced-learn** - SMOTE for class balancing
- **pyautogui** - Mouse tracking
- **tkinter** - GUI framework
- **joblib** - Model persistence

Usage

1. Data Collection

```
# In the GUI:  
1. Enter username (e.g., "Jonathan")  
2. Click "🔗 Collect Training Data (4 Sessions)"  
3. Follow the 10 red dots smoothly with your mouse  
4. Complete all 4 sessions  
5. Repeat for 2-3 more users
```

Tips for Best Results:

- ☒ Move smoothly and naturally
- ☒ Use consistent speed
- ☒ Follow curved paths
- ☒ Avoid jerky movements
- ☒ Don't move too fast

2. Model Training

```
# In the GUI:  
1. Click "🎓 Train Model"  
2. Wait for training to complete (10-30 seconds)  
3. Review accuracy metrics and per-user performance  
4. Click "💾 Save" to persist the model
```

3. Authentication

```
# In the GUI:  
1. Select security level:  
  - Low: 40% confidence, 50% consistency  
  - Medium: 55% confidence, 65% consistency  
  - High: 70% confidence, 80% consistency  
  
2. Click "🔑 Authenticate User"  
3. Follow the 10 dots naturally  
4. System will:  
  ☒ Grant access if all checks pass  
  ☒ Deny access and show predicted user
```

Technical Specifications

Machine Learning Pipeline

Model Architecture

```
Voting Classifier (Soft Voting)
├── XGBoost (Weight: 1.5)
│   ├── Objective: Multi-class softmax
│   ├── Learning Rate: 0.03
│   ├── Estimators: 200
│   ├── Max Depth: 5
│   ├── Regularization: L1=1.0, L2=2.0
│   └── Early Stopping: Enabled
└── Random Forest (Weight: 1.0)
    ├── Estimators: 75
    ├── Max Depth: 6
    ├── Class Weight: Balanced
    └── Parallel Jobs: All CPUs
```

Data Processing

- **Train/Test Split:** 70/30 stratified
- **Feature Scaling:** StandardScaler (zero mean, unit variance)
- **Class Balancing:** SMOTE when imbalance ratio > 2:1
- **Cross-Validation:** 3-fold stratified

Authentication Algorithm

```
overall_confidence = (
    0.30 × mean_sample_confidence +
    0.25 × vote_consistency +
    0.20 × median_confidence +
    0.15 × min_sample_confidence +
    0.10 × top2_margin
)

authentication_passed = (
    overall_confidence ≥ threshold AND
    vote_consistency ≥ consistency_threshold AND
    min_confidence ≥ (threshold - 0.15)
)
```

 Performance Metrics

Academic Metrics (for reporting)

Metric	Formula	Purpose
Accuracy	Correct predictions / Total predictions	Overall system performance
FAR	False Acceptances / Total Impostor Attempts	Security (lower is better)

Metric	Formula	Purpose
FRR	False Rejections / Total Genuine Attempts	Usability (lower is better)
EER	Point where FAR = FRR	Overall balance point

System Performance Indicators

Training Metrics:

└─ Training Accuracy: Model fit quality

└─ Test Accuracy: Generalization capability

└─ Cross-Validation: Stability across folds

└─ Overfitting Gap: Train - Test accuracy

Per-User Metrics:

└─ Individual Accuracy: User-specific performance

└─ Sample Count: Training data per user

└─ Pattern Consistency: Speed variation coefficient

Authentication Metrics:

└─ Overall Confidence: Weighted multi-metric score

└─ Vote Consistency: Prediction agreement %

└─ Min Sample Confidence: Worst-case scenario

└─ Consistency Score: $1 - (\text{std_dev} / 0.3)$

Project Structure

Project/

└─ Mouse-Recognition-FIXED.py

└─ run.bat

└─ README.md

└─ mouse_features.csv

└─ mouse_auth_model.pkl

└─ mouse/

└─ python.exe

└─ Lib/site-packages/

└─ CSV_TEST/

└─ 1/

└─ 4 Person/

Main application

Environment launcher

This file

Training data

Saved model (generated)

Conda environment

Test datasets

User Interface

Dark Mode Theme

- Background: #0f172a (Dark Navy)
- Cards: #1e293b (Dark Slate)

- Primary: #3b82f6 (Bright Blue)
- Success: #10b981 (Green)
- Warning: #f59e0b (Orange)
- Danger: #ef4444 (Red)

Main Sections

1. **Data Collection** - 10-dot pattern tracking
2. **Model Management** - Train/Save/Load
3. **Authentication** - Real-time verification
4. **Security Levels** - Adjustable thresholds
5. **Analytics** - Data quality & performance reports

Research Applications

This system demonstrates:

- ☒ Behavioral biometrics feasibility
- ☒ Continuous authentication viability
- ☒ Machine learning for security
- ☒ Real-time pattern recognition
- ☒ Multi-user classification

Suitable For:

-  Academic research projects
-  Secondary authentication layer
-  Workstation access control
-  Shared computer environments
-  Behavioral analysis studies

Limitations & Future Work

Current Limitations

- Windows-only (PyAutoGUI implementation)
- Requires consistent mouse usage
- Performance varies by user (91% to 2%)
- No click/scroll dynamics captured
- Static 10-dot pattern

Proposed Enhancements

1. **Multi-Modal Biometrics:** Add keystroke dynamics
2. **Advanced Features:** Click timing, scroll patterns, drag-drop
3. **Adaptive Learning:** Online updates for concept drift
4. **Deep Learning:** LSTM/Transformer for temporal sequences
5. **Privacy:** Differential privacy mechanisms

6. **Cross-Platform:** macOS and Linux support

Contributors

- **Jonathan** - Project Lead & Development
 - **Dr. Hisham** - Academic Supervision
 - **Mariam** - Testing & Validation
-

License

This project is part of an academic research study for the Cyber Physical Systems course at AAST.

Support

For questions or issues:

1. Check **Data Quality Report** in the app
 2. Review training metrics for accuracy issues
 3. Ensure smooth mouse movements during collection
 4. Try different security levels (Low for testing)
-

Built with  **using Python, XGBoost, and Scikit-learn**